

HW4 –Support Vector Machine Report

1. 資料前處理說明

1.1. 使用了什麼樣的前處理方式

對非數值而是文字表示的資料欄位：性別 (Gender) 與國家 (Geography)，需要先將他們轉成數字，讓 SVM 可以理解：

- **性別 (Gender)**：Female 轉成 0、Male 轉成 1。
- **國家 (Geography)**：France 轉成 [1,0,0]、Germany 轉成 [0,1,0]、Spain 轉成 [0,0,1]。

對數值化的資料，我主要對資料做了兩個步驟的前處理：

- **Step1. 標準化**：將每個特徵都調整成平均值是 0、標準差是 1，讓不同單位的欄位（像年齡、薪水等等）可以在同一個尺度上比較。
- **Step2. 正規化**：將每一筆標準化後的資料的整個向量長度調成 1，使重點聚焦在每個樣本之間的方向，使其不會被數值大小影響。

另外，在驗證資料 (validation) 跟測試資料 (test) 上，也是做一樣的處理，但最大的差異是這兩組資料在標準化的時候，並不是採用它們 (validation 和 test) 本身的平均值跟標準差去處理，而是直接用 train 資料的統計值來轉換，是為了避免讓模型偷看到原本不應該看到的資訊，而造成資料洩漏、使準確率

看起來變高但其實不準的問題。

1.2. 資料視覺化圖表

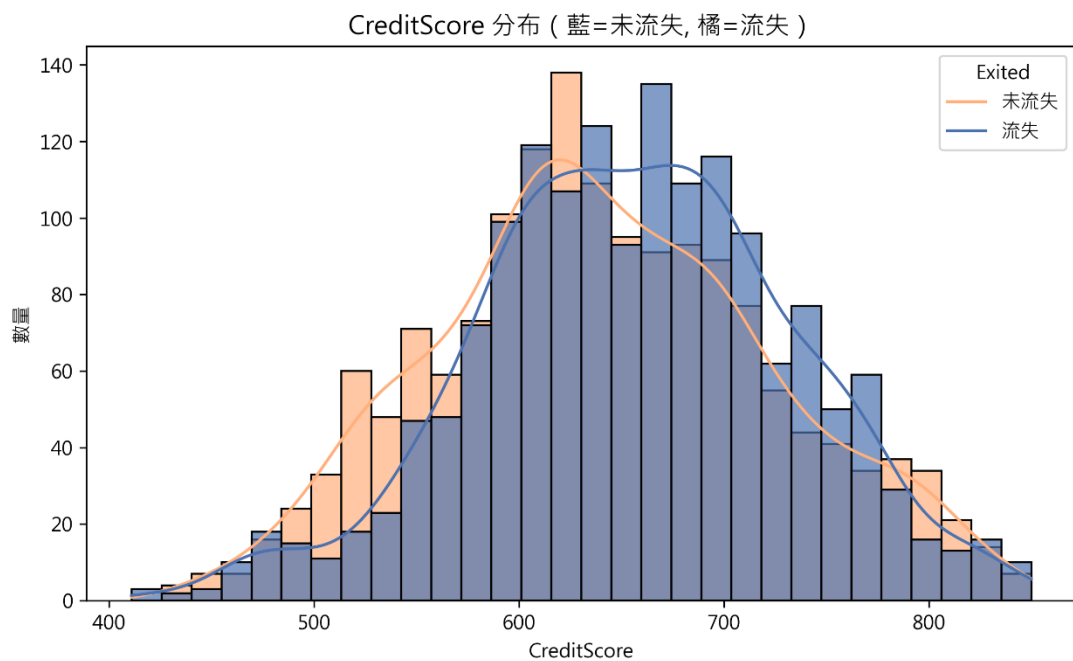
一開始先將明顯無關的項目欄位刪除，如：RowNumber (資料編號)、

CustomerId (客戶編號)、Surname (客戶姓氏)。

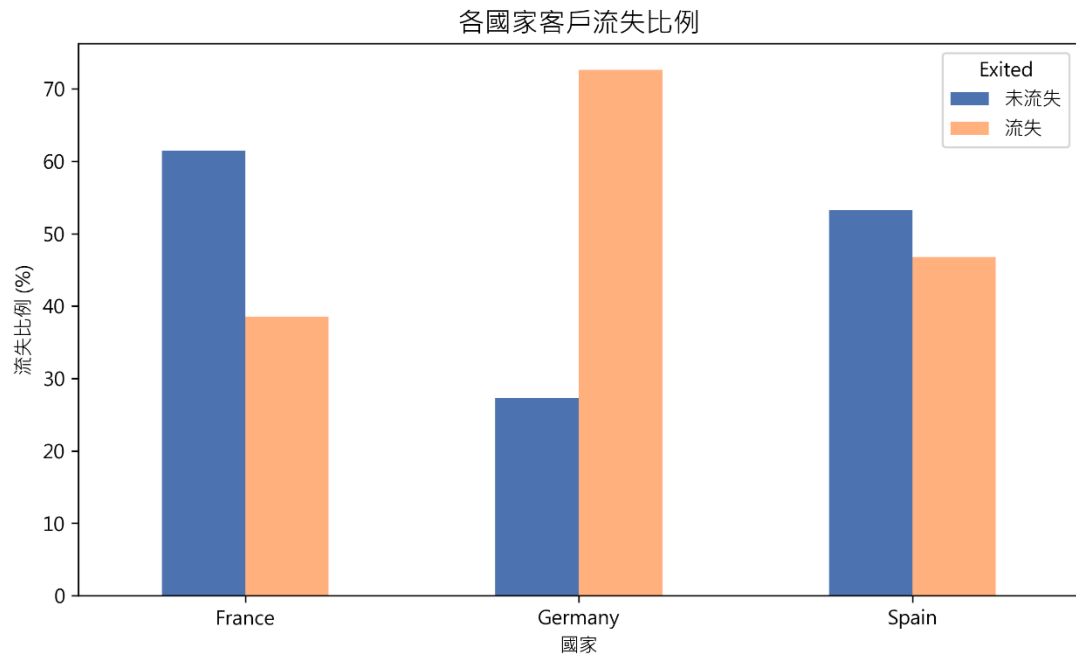
接著再以 Python 程式對 train 資料剩餘的項目欄位特徵分別與 Exited 的相關性

進行資料視覺化分析：

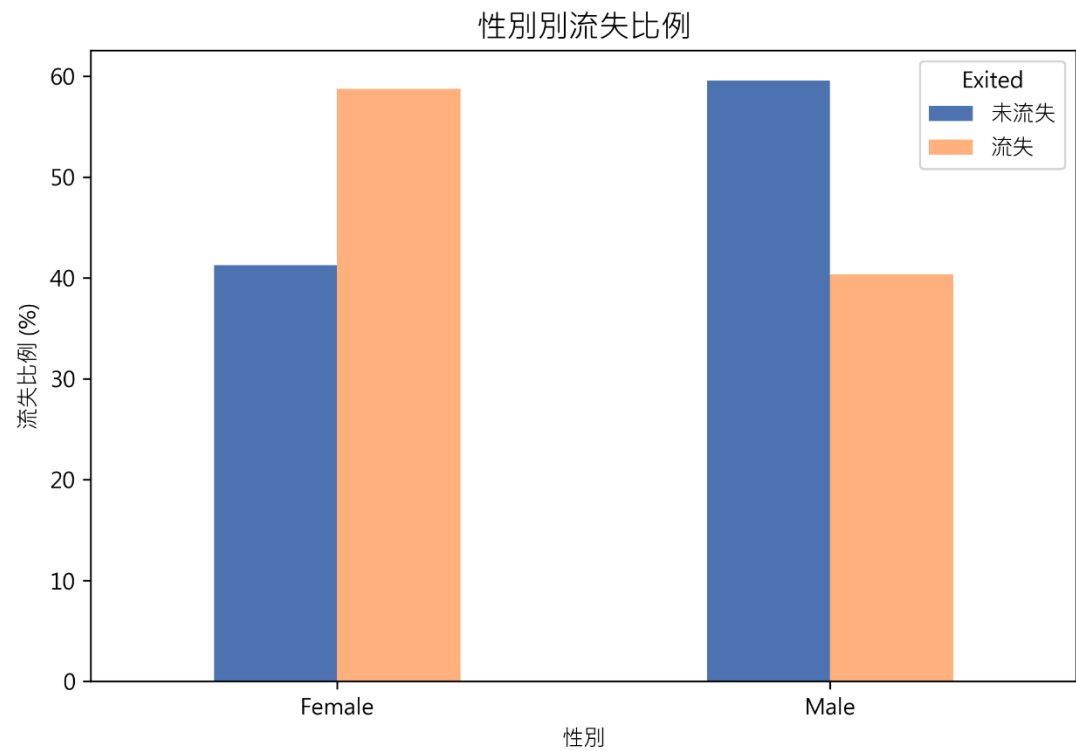
◆ 關於 CreditScore (信用評分) 欄位 → 明顯有差距，採用



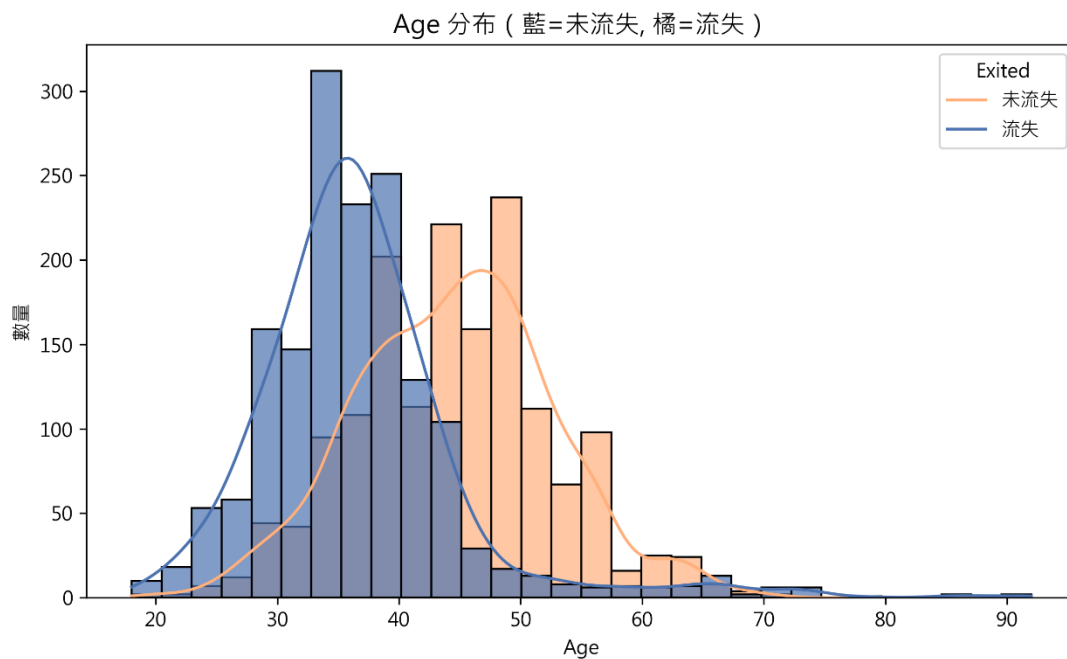
◆ 關於 Geography (客戶所在國家) 欄位 → 明顯有差距，採用



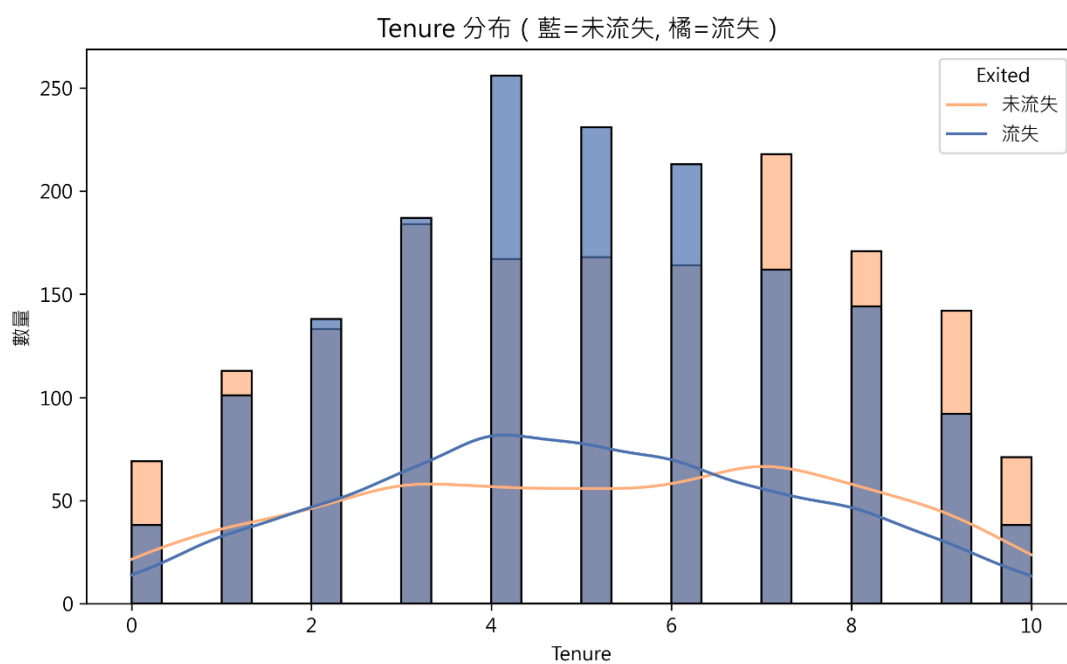
◆ 關於 Gender (性別) 欄位 → 明顯有差距，採用



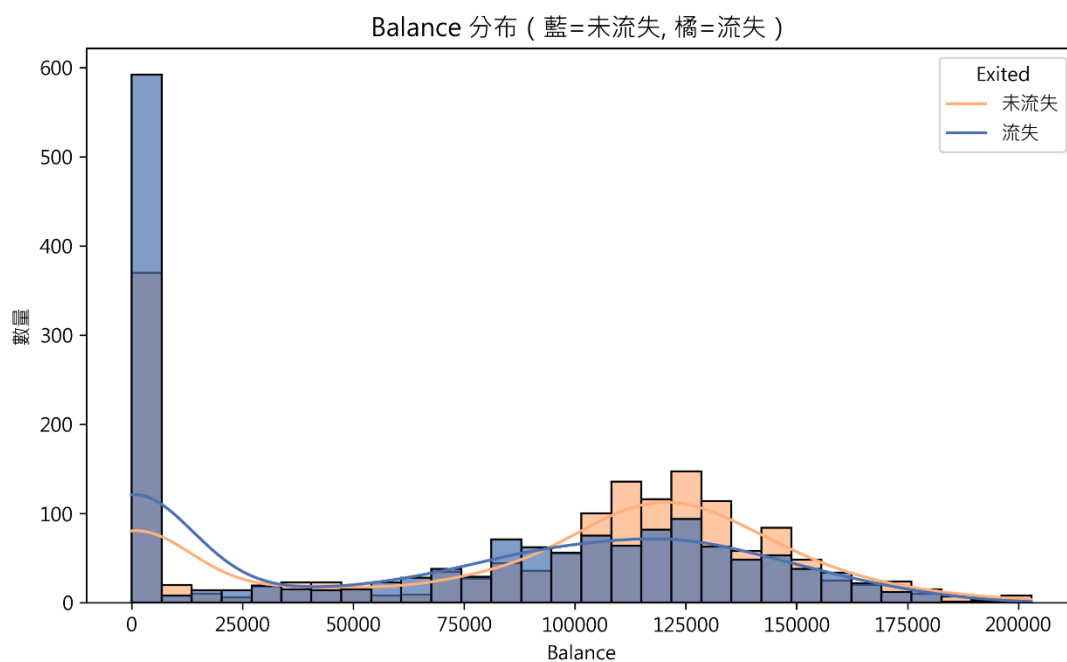
◆ 關於 Age (年齡) 欄位 → 明顯有差距，採用



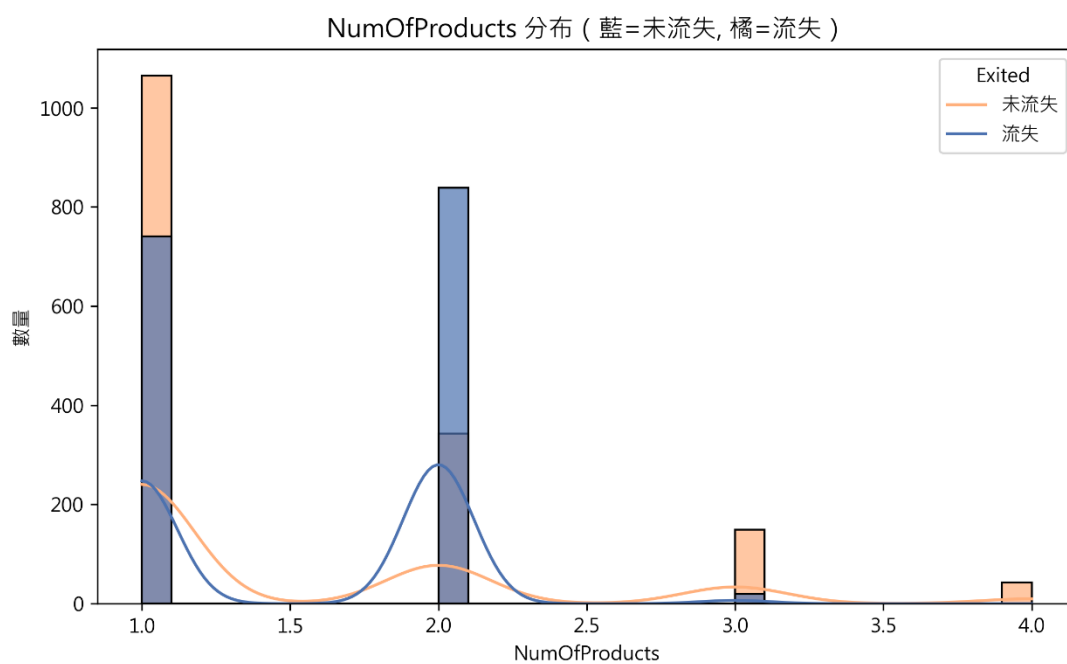
◆ 關於 Tenure (客戶在銀行的年資) 欄位 → 明顯有差距，採用



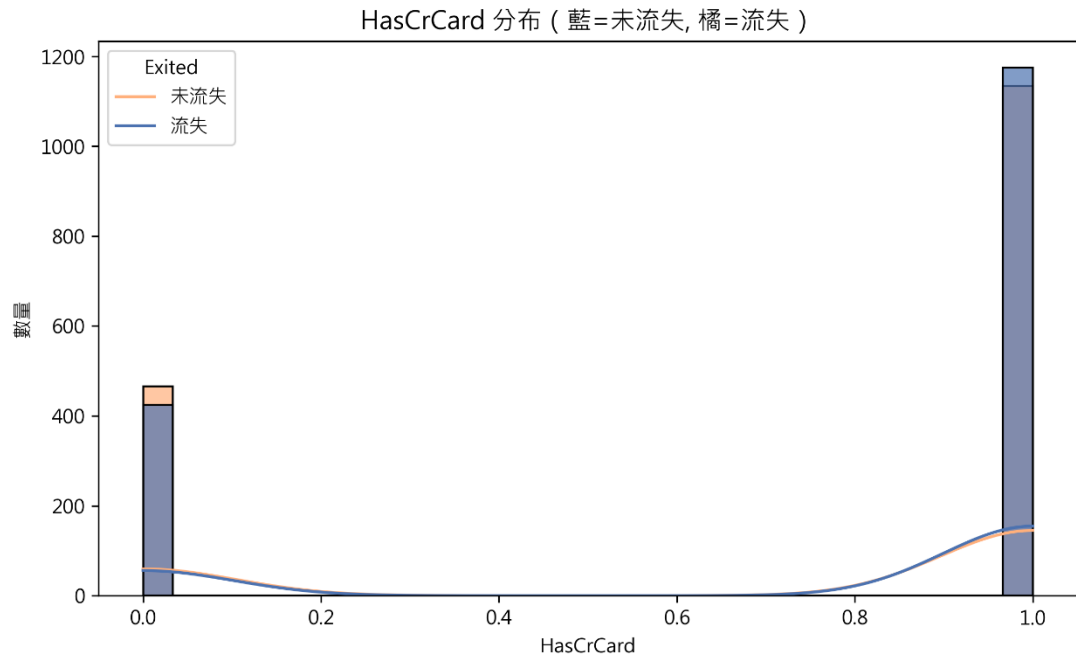
◆ 關於 Balance (銀行帳戶餘額) 欄位 → 明顯有差距，採用



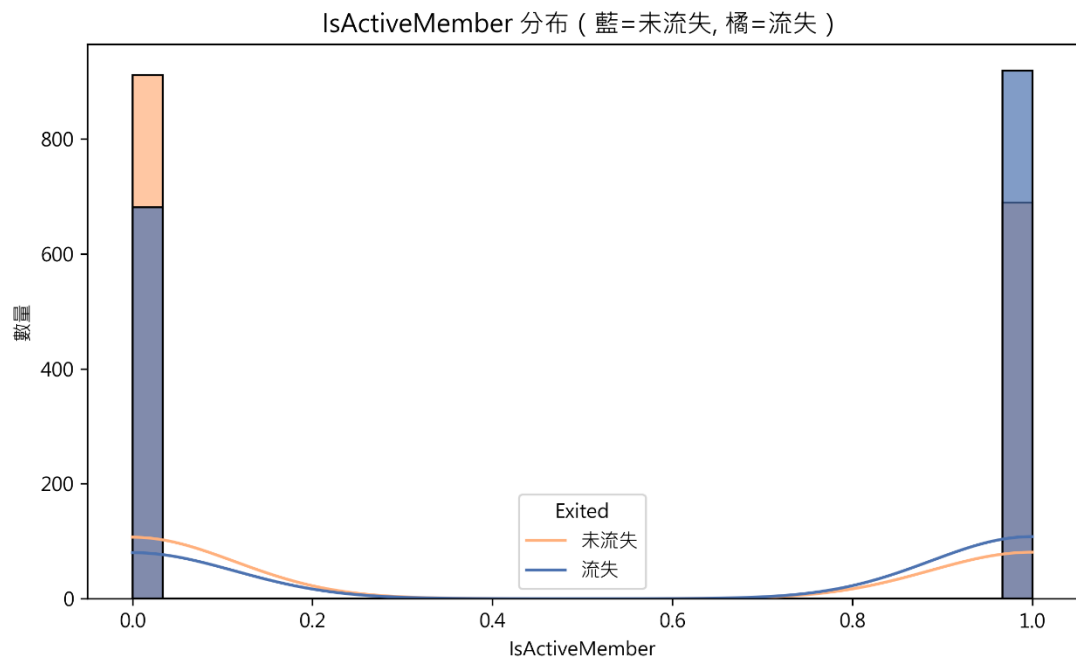
◆ 關於 NumOfProducts (使用的銀行產品數) 欄位 → 明顯有差距，採用



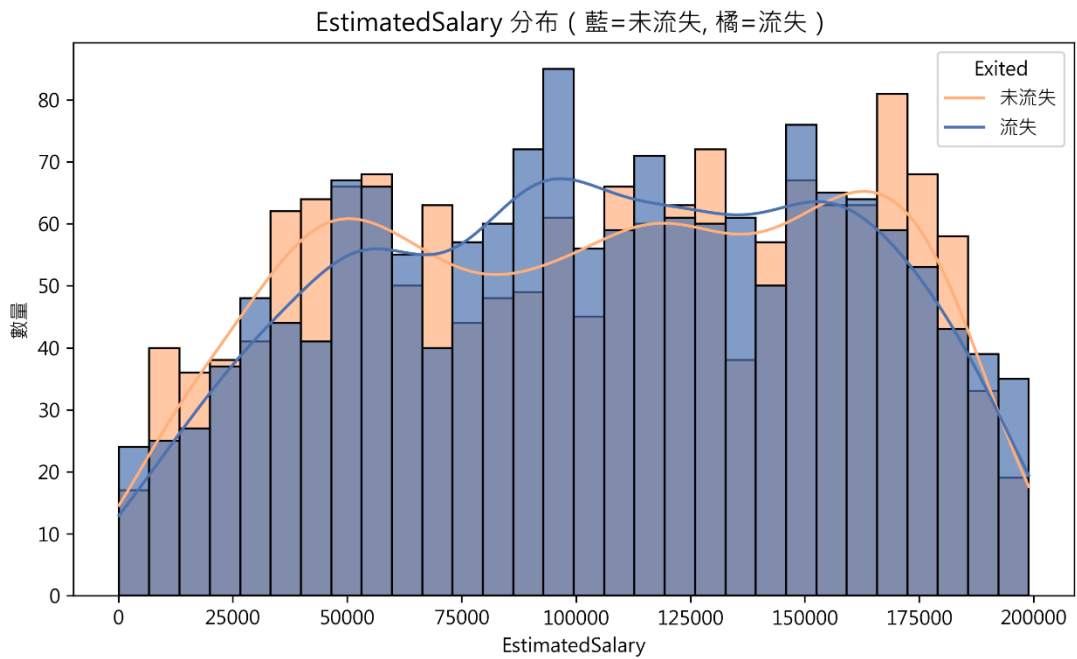
◆ 關於 HasCrCard (是否擁有信用卡) 欄位 → 微差距，但怕漏掉還是採用



◆ 關於 IsActiveMember (是否為活躍客戶) 欄位 → 明顯有差距，採用



◆ 關於 EstimatedSalary (客戶估計年薪) 欄位 → 明顯有差距，採用



1.3. 最後挑選了哪些欄位作為輸入

下表為最終使用的特徵欄位與對應意義：

欄位名稱	說明
CreditScore	信用評分
Geography	客戶所在國家
Gender	性別
Age	年齡
Tenure	客戶在銀行的年資
Balance	銀行帳戶餘額
NumOfProducts	使用的銀行產品數
HasCrCard	是否擁有信用卡

IsActiveMember	是否為活躍客戶
EstimatedSalary	客戶估計年薪

2. SVM 模型選擇說明

2.1. 怎麼挑選 SVM 的參數？

我使用的模型是 Polynomial Kernel 的 SVM：

- Other kernel functions: $K(\mathbf{x}_j, \mathbf{x}_k) = (1 + \mathbf{x}_j \cdot \mathbf{x}_k)^d$

核函数	计算公式
线性核	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
多项式核	$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + b)^d$
径向基函数核/高斯核	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$
sigmoid 核	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^T \mathbf{x}_j + b)$

嘗試不同的參數組合後，最後找到一組表現最穩定、準確率最高的設定：

```
# =====Polynomial kernel SVM 相關參數設定=====
svm = SVC(
    kernel="poly", # 使用多項式核
    degree=7, # 多項式的次方
    gamma=1.0, # xz的放大尺度
    coef0=1.0, # 多項式中的常數項
    C=1.0, # SVM 中的懲罰參數
    random_state=40, # 固定輸出
)
```

2.2. 為甚麼這樣挑選參數的效果會比較好或比較不好？

主要調整的參數是 `degree` 和 `gamma` 還有 `coef0`，選擇上述的參數理由如下：

當 `degree` 太小 (像 1 或 2) 時，模型太簡單，只能分出線性關係，準確率偏低；而當 `degree` 太大 (像 9 以上) 時，模型會太複雜，反而會造成過擬合的情況；`degree=7` 是一個剛好的折衷，能抓住非線性關係但又不會太亂。

再來是 `gamma` 的部分，在其他參數不變情況下，當它設成 "scale" 或 1.0 時，準確率可以到 1.0；但如果設成 "auto"，準確率就會掉到 0.853。原因是 "auto" 會讓模型的「影響範圍」太小，SVM 的邊界會變得太硬、太侷限，沒辦法有效捕捉資料之間的曲線關係，因此最後選擇簡單且表現優良的 1.0 作為 `gamma` 的參數值。

`coef0` 控制多項式核中「低次項與高次項」的權重比例。理論上，較大的 `coef0` 會使模型傾向產生較平滑的邊界；而較小的值則使模型更依賴高次項，邊界更複雜。在實驗中，當 `coef0` 設為 1.0、2.0、3.0 時，皆可達到高準確率 (約 0.99 ~ 1.0)，差異不大。但觀察到 `coef0 = 1.0` 在不同次方設定下表現更穩定，且運算複雜度較低，因此最終選擇 `coef0 = 1.0` 作為最簡潔且穩定的設定值。

2.3. 比較兩種以上不同參數的 SVM 帶來的結果，並說明模型產生不同結果的原因

主要做了幾組不同的實驗組合：以 polynomial 核作為 SVM 提升預測準確值的過程中，曾針對 gamma 的數值、coef0 常數項和 degree 值做組合參數變化以觀察預測準確值的變化。

■ 在其餘參數不變的情況下，針對不同的 gamma 數值並對 degree 作變化：

其餘參數不變 (`kernel="poly"` , `degree=7` , `coef0=1.0` , `C=1.0` , `random_state=40`)：

將 gamma 參數值改成 "scale" 和 "1.0" 時 (預測出的 validation 準確率皆為 1.0)，表現會比 gamma 參數值為 "auto" 好很多 (預測出的 validation 準確率約為 0.853)。

P.S. gamma = "auto" 並且如果希望將預測出的 validation 準確率提升約為 0.998，在只更動 degree 參數的情況下需要將 degree 提升至 28 次方。

P.S. gamma = "scale" 和 gamma = 1.0 在其他各項參數不變只改變 degree 數值的情況下，值的差異不大 (小於 0.005)。

→ 原因：gamma 會決定樣本之間「相似度」衰減的速度，也就是每一個點對決策邊界的影響範圍。而 gamma = "auto" 的取值方式會讓邊界太僵化、過度依賴局部訊息，對小變化太敏感，反而無法學習整體趨勢。相對地，使用

"scale" 或固定成 1.0，在這個資料集上可以讓模型的影響範圍處在比較合理

的區間，既不會對資料太敏感，也不會太遲鈍，能穩定學到整體的模式，所以

準確率比較好。

■ 在其餘參數不變的情況下，針對不同的 `coef0` 數值對 `degree` 作變化：

其餘參數不變 (`kernel="poly"` , `gamma=1.0` , `C=1.0` , `random_state=40`)：

	coef0 = 1.0	coef0 = 2.0	coef0 = 3.0
degree = 1	預測出的準確率 =0.752	預測出的準確率 =0.752	預測出的準確率 =0.752
degree = 2	0.853	0.853	0.853
degree = 3	0.958	0.958	0.950
degree = 4	0.993	0.998	1.0
degree = 5	0.998	1.0	0.998
degree = 6	0.998	0.995	0.988
degree = 7	1.0	0.995	0.990
degree = 8	1.0	0.998	0.995
degree = 9	0.998	0.998	0.995
degree = 10	0.998	0.998	0.998
degree = 11	0.998	0.998	0.998

→ 原因：`coef0` 是多項式核裡的常數項，主要負責調整「低次項 vs 高次項」

的比重。從表格可以看到，當 `coef0 = 1.0`、`2.0`、`3.0` 時，整體準確率差異非常

小 (多數維持在 0.99 左右)，而且將 `coef0` 提高並沒有顯著改善準確率，甚

至在某些次方下略微下降 (如 `degree = 7` 時，`coef0 = 1.0` 的表現最佳)。

說明在這個資料集上：

模型的非線性學習能力主要受到 **degree** 控制，而 **coef0** 的影響相對次要。

因此，為了避免模型邊界過於平滑或複雜（當 **coef0** 較小（例如 0 或 1）時主要
由多項式中的高次項決定 → 強調「特徵間的高階交互作用」，學習邊界能較
複雜；而當 **coef0** 較大（例如 2、3、5）時，低次項和常數項的比重則會變
高 → 會更快趨近穩定（例如訓練準確率很快提升），但實際上是因為模型邊界
變得平滑、非線性成分減弱。而關於在本資料中，由於主要的非線性特徵已可
由 **degree** 捕捉，因此 **coef0** 的影響相對次要。綜合考量穩定性與模型複雜度
後，最終選擇 **coef0 = 1.0** 作為簡潔且穩定的設定。

■ 在其餘參數不變的情況下，只針對 **degree** 值作變化：

其餘參數不變（**kernel="poly"**，**gamma=1.0**，**coef0=1.0**，**C=1.0**，

random_state=40）：

1. 當 **degree** 值從 1 慢慢增加到 7 的時候，預測出的 validation 準確率會逐步穩定提升，當 **degree** 值到 7、8 的時候，預測出的 validation 準確率會達到 1.0。
2. 當 **degree** 值超過 8（例如：9~16）的時候，預測出的 validation 準確率會一直維持在 0.998。

(雖然 $\text{degree} = 8$ 的模型會比 $\text{degree} = 7$ 的模型來的稍微複雜、計算量也變大了些，所以最後我選擇了 $\text{degree}=7$ 這個比較穩定的設定。)。

→ 原因： degree 決定了模型可以畫出多曲折、複雜的邊界。一開始在 degree 很小 (1、2、3) 的時候，模型的表現能力還不足夠，而隨著 degree 增加，模型學到的非線性關係變多，因此準確率逐漸提升。當 degree 大概到 7、8 時，模型的複雜度已經足夠去描述資料的主要結構，再提高至 9~16，只是讓模型有能力去處理更小的細節，雖表現仍維持高準確率 (約 0.998 ~ 1.0)，但同時增加了模型的計算成本與過擬合風險。所以最終選擇 $\text{degree} = 7$ 這個比較簡單可靠、表現好的模型設定參數。